

# POKÉMON BATTLE

*Plataforma Distribuida de Batallas por Turnos*

---

## Documento de Sustentación Técnica

---

Tecnología: Elixir / OTP / GenServer / Erlang Distribution

Versión 1.0 · 2026

# 1. Introducción General al Proyecto

PokémonBattle es una plataforma de combates Pokémon 1v1 por turnos desarrollada completamente en Elixir, aprovechando el ecosistema de Erlang/OTP para lograr concurrencia real, distribución entre múltiples nodos y tolerancia a fallos. El proyecto nació como ejercicio de programación funcional y concurrente, pero creció hasta convertirse en un sistema completo con economía virtual, intercambio de Pokémon en tiempo real y evolución automática de personajes.

El sistema corre en consola interactiva (IEx) y permite que dos jugadores conectados desde terminales completamente distintas, incluso en máquinas físicas diferentes conectadas por red, puedan jugar una batalla en tiempo real donde cada uno ve los eventos en su propia pantalla y responde con sus propios comandos de forma simultánea.

## 1.1 Objetivos cumplidos

- Batallas 1v1 simultáneas en múltiples salas de forma concurrente sin interferencia entre ellas.
- Ejecución distribuida en al menos 2 nodos Erlang conectados por red.
- Motor de combate por turnos con efectividad de tipos, STAB y factor aleatorio.
- Sistema de economía: monedas, compra y apertura de sobres, inventario persistente.
- Intercambio de Pokémon en tiempo real entre jugadores de distintos nodos.
- CRUD completo de equipos: crear, ver, renombrar, agregar, quitar y eliminar.
- Sistema de evolución automática tras victorias acumuladas en batalla.
- Persistencia de todos los datos en archivos JSON entre sesiones.
- Detección de desconexión en batalla con ventana de 15 segundos para reconectar.

## 1.2 Temas del curso implementados

| Tema del curso                               | Dónde se implementa en el proyecto                                                  |
|----------------------------------------------|-------------------------------------------------------------------------------------|
| Paradigmas de programación funcional         | Todos los módulos: datos inmutables, funciones puras, sin estado mutable compartido |
| Introducción a Elixir y Mix                  | mix.exs, Application, estructura de proyecto                                        |
| Expresiones lógicas y estructuras de control | cond/case/if en Batalla, GestorSalas, GestorEntrenadores                            |
| Colecciones en Elixir                        | Enum, Map, List en todos los módulos para manejar equipos, inventarios y equipos    |
| Recursividad                                 | Evolution.build_chain/3 recorre cadenas de evolución recursivamente                 |
| Structs                                      | PokemonBattle.Servidor usa defstruct para la sesión del jugador                     |

| Tema del curso            | Dónde se implementa en el proyecto                                   |
|---------------------------|----------------------------------------------------------------------|
| Manejo de archivos        | Persistencia lee y escribe JSON con File.read/write y Jason          |
| Procesos en Elixir        | send/receive para notificaciones de batalla entre procesos distintos |
| OTP y GenServer           | Batalla, GestorSalas, Intercambio son GenServers completos           |
| DynamicSupervisor         | SupervisorBatallas supervisa batallas e intercambios dinámicamente   |
| Aplicaciones distribuidas | Cluster conecta nodos, RPC enruta acciones entre node1 y node2       |

## 2. Programación Funcional en Elixir

Todo el proyecto está construido bajo el paradigma funcional puro de Elixir. Esto significa que no existe ninguna variable mutable global, ningún objeto que cambie su estado interno, y ningún efecto secundario oculto. Cada función recibe datos, los transforma y retorna nuevos datos, siempre dejando los originales intactos.

### 2.1 Inmutabilidad y transformación de datos

El ejemplo más claro está en la resolución de un turno de batalla. El estado de la batalla es un mapa inmutable. Cada acción genera un nuevo mapa con los cambios aplicados, sin modificar el anterior:

```
state1 = execute_action(state, first, second, act_first)
state2 = if active_fainted?(state1, second), do: state1,
         else: execute_action(state1, second, first, act_second)
state3 = check_end(state2)
```

Cada línea produce un nuevo estado. Si algo falla en state2, state1 sigue intacto. Esto hace el sistema predecible y fácil de razonar.

### 2.2 El operador pipe |>

Elixir permite encadenar transformaciones de forma legible usando el operador pipe, que pasa el resultado de una expresión como primer argumento de la siguiente:

```
trainer_with_wins =
  t
  |> Map.put("inventory", updated_inventory)
  |> Map.put("wins", t["wins"] + 1)
  |> Map.put("coins", t["coins"] + 100)
  |> Map.put("accumulated_coins", t["accumulated_coins"] + 100)
  |> Evolution.check_evolutions()
```

Esto es equivalente a llamar cada función anidada, pero se lee de arriba a abajo como una secuencia lógica de pasos.

### 2.3 Pattern matching

El pattern matching es el mecanismo central de Elixir para tomar decisiones. En lugar de if/else con condiciones booleanas, las funciones se definen con múltiples cláusulas que se activan según la forma del dato recibido:

```
def handle_call(:join, trainer, team, pid, _from, state) do ... end
def handle_call(:start, _from, state) do ... end
def handle_call(:action, username, action, _from, state) do ... end
```

El runtime de Erlang selecciona automáticamente la cláusula correcta según el patrón del mensaje recibido. Esto elimina grandes bloques de if anidados y hace el código más expresivo.

## 2.4 Enum y colecciones funcionales

Todas las operaciones sobre listas usan funciones de orden superior del módulo Enum, que reciben funciones como argumentos:

```
# Actualizar un Pokemon en el equipo:
new_team = Enum.map(state.teams[username], fn p ->
  if p["id"] == updated_pokemon["id"], do: updated_pokemon, else: p
end)
```

```
# Verificar si todos los Pokemon están debilitados:
defp all_fainted?(team), do: Enum.all?(team, &MotorCombate.fainted?/1)
```

```
# Filtrar jugadores para encontrar al rival:
[winner] = Map.keys(state.players) |> Enum.reject(&(&1 == username))
```

## 3. Procesos en Elixir y OTP

La concurrencia en Elixir se basa en procesos ligeros del runtime de Erlang, completamente aislados entre sí. No comparten memoria: se comunican exclusivamente por mensajes. Esto es lo que permite que múltiples batallas corran al mismo tiempo sin interferirse.

### 3.1 ¿Qué es un proceso en Elixir?

Un proceso de Elixir no es un hilo del sistema operativo (thread). Es una unidad de ejecución manejada por la máquina virtual BEAM, extremadamente liviana. Se pueden tener millones de procesos corriendo simultáneamente en un solo servidor. Cada proceso tiene su propio buzón de mensajes (mailbox) donde recibe mensajes de otros procesos.

### 3.2 GenServer — el corazón del sistema

GenServer (Generic Server) es un comportamiento de OTP que encapsula un proceso con estado de forma estructurada. Cada batalla es un GenServer independiente con su propio estado:

```
defmodule PokemonBattle.Batalla do
  use GenServer

  # Estado inicial de la batalla
  def init(opts) do
    state = %{
      room_id: Keyword.fetch!(opts, :room_id),
      players: %{},
      actives: %{},
      actions: %{},
      pids: %{}, # PIDs de sesión de cada jugador
      phase: :waiting,
      turn: 0,
      ...
    }
    {:ok, state}
  end
end
```

El flujo de un GenServer siempre es el mismo:

- Un proceso externo llama a una función pública (join, send\_action, etc.).
- Esa función envía un mensaje al proceso GenServer con GenServer.call/2.
- El GenServer recibe el mensaje en handle\_call/3, procesa la lógica y retorna una respuesta.
- El estado nuevo queda guardado en el proceso hasta el próximo mensaje.

### 3.3 Llamadas síncronas vs mensajes asíncronos

El sistema usa dos formas de comunicación entre procesos:

| Mecanismo                     | Cuándo se usa                                  | Ejemplo en el proyecto                       |
|-------------------------------|------------------------------------------------|----------------------------------------------|
| GenServer.call/2 (síncrono)   | Cuando se necesita una respuesta inmediata     | attack, join_room, confirm_trade             |
| send/2 (asíncrono)            | Para notificaciones que no requieren respuesta | Eventos de batalla a terminales de jugadores |
| :rpc.call/4 (remoto síncrono) | Llamadas a funciones en otro nodo del cluster  | Luis en node2 ataca, la llamada va a node1   |

### 3.4 El sistema de notificaciones por PID

Este es el diseño más importante del proyecto para lograr las batallas simultáneas. Cuando un jugador hace start() en su terminal, ese proceso tiene un PID único. Al unirse a una sala, ese PID se registra en el estado del GenServer de batalla:

```
# En batalla.ex – estado tiene un mapa de PIDs
pids: %{"ana" => #PID<0.234.0>, "luis" => #PID<1.456.0>}
```

```
# Al ocurrir un evento, se envía mensaje al PID de cada jugador
def broadcast(state, message) do
  Enum.each(state.pids, fn {_username, pid} ->
    send(pid, {:battle_event, message})
  end)
end
```

```
# En servidor.ex – el loop drena los mensajes antes del prompt
defp flush_battle_events do
  receive do
    {:battle_event, msg} ->
      IO.puts(msg)
      flush_battle_events()
  after
    0 -> :ok # Si no hay mensajes, continúa inmediatamente
  end
end
```

Esto garantiza que la terminal de Ana solo muestra los mensajes enviados al PID de Ana, y la de Luis solo los suyos. Ambos pueden escribir sus acciones de forma completamente independiente.

## 4. Supervisión y Tolerancia a Fallos

OTP provee una filosofía de diseño llamada "Let it crash": en lugar de intentar manejar todos los errores posibles dentro de cada proceso, se deja que los procesos fallen y se confía en que un supervisor los reinicie o tome acción. Esto hace los sistemas más robustos que los que intentan manejar todos los casos de error.

### 4.1 Árbol de supervisión del proyecto

Cuando la aplicación arranca, se inicia un árbol jerárquico de supervisores y procesos:

```
PokemonBattle.Supervisor (estrategia: one_for_one)
├─ Registry                (registro global de procesos por nombre)
├─ SupervisorBatallas      (DynamicSupervisor)
│   ├─ Batalla <S-0001>    (GenServer – batalla activa)
│   ├─ Batalla <S-0002>    (GenServer – otra batalla simultánea)
│   └─ Intercambio <TR-1234> (GenServer – sala de trade)
└─ GestorSalas             (GenServer – solo en nodo primario)
```

### 4.2 DynamicSupervisor — supervisión dinámica

A diferencia de un Supervisor normal donde los procesos hijos se definen en tiempo de compilación, el DynamicSupervisor permite agregar y eliminar procesos hijos en tiempo de ejecución. Esto es perfecto para batallas porque no sabemos cuántas habrá ni cuándo empezarán:

```
def start_battle(room_id, turn_time \\ 20) do
  spec = {PokemonBattle.Batalla, [room_id: room_id, turn_time: turn_time]}
  DynamicSupervisor.start_child(__MODULE__, spec)
end
```

Cuando se crea una sala y dos jugadores inician la batalla, el DynamicSupervisor lanza un nuevo proceso GenServer de Batalla. Si ese proceso falla por algún error inesperado, solo esa batalla se ve afectada. Las demás batallas que corren en paralelo continúan sin interrupción.

### 4.3 Registry — localizar procesos por nombre

El Registry permite registrar procesos con un nombre lógico y encontrarlos después sin necesidad de guardar el PID manualmente. Esto es crítico en un sistema distribuido donde los PIDs pueden cambiar:

```
# Registrar la batalla con su room_id como nombre
defp via(room_id) do
  {:via, Registry, {PokemonBattle.Registry, {:battle, room_id}}}
end
```



```
# Llamar a la batalla por nombre, sin conocer el PID
def send_action(room_id, username, action) do
  GenServer.call(via(room_id), {:action, username, action})
end
```

## 4.4 Estrategia one\_for\_one

La estrategia one\_for\_one significa que si un proceso hijo falla, solo ese proceso se reinicia. Los demás hijos no se ven afectados. En el contexto del proyecto, si una batalla falla, las demás batallas activas continúan normalmente, y el GestorSalas sigue funcionando para crear nuevas salas.

## 5. Aplicaciones Distribuidas — Cluster Erlang

Erlang tiene distribución integrada en el runtime. Dos nodos de Erlang/Elixir pueden conectarse por red y enviar mensajes entre procesos como si estuvieran en la misma máquina. El sistema PokémonBattle aprovecha esto para distribuir batallas entre nodos.

### 5.1 Cómo se conectan los nodos

Cada nodo tiene un nombre único y una cookie compartida que actúa como contraseña de autenticación:

```
# Terminal 1 – Nodo primario
iex --sname node1 --cookie pokemonbattle -S mix
```

```
# Terminal 2 – Nodo secundario
iex --sname node2 --cookie pokemonbattle -S mix
```

```
# Conectar desde node2 a node1
PokemonBattle.Cluster.connect("node1@hostname")
```

Una vez conectados, ambos nodos forman un cluster y pueden verse mutuamente. `Node.list()` retorna la lista de nodos conectados desde cualquiera de los dos.

### 5.2 Nodo primario vs nodo secundario

El diseño del proyecto diferencia entre nodo primario y secundario. El `GestorSalas` (que maneja las salas de espera) solo corre en el nodo primario, identificado porque su nombre contiene "node1":

```
def start(_type, _args) do
  children = [Registry, SupervisorBatallas]

  children = if Node.self() == :nonode@nohost or primary_node?() do
    children ++ [GestorSalas] # Solo en nodo primario
  else
    children # Nodo secundario sin GestorSalas
  end
  Supervisor.start_link(children, strategy: :one_for_one)
end
```

### 5.3 RPC — Llamadas remotas a procedimientos

Cuando Luis está en node2 y necesita realizar una acción que vive en node1 (como atacar en una batalla que corre allá), el Servidor usa `:rpc.call` para ejecutar la función en el nodo remoto y recibir el resultado:

```
defp route_battle_action(room_id, action) do
  primary = get_primary_node()
  if primary == Node.self() do
    # Estamos en node1, llamada local
    PokemonBattle.Batalla.send_action(room_id, username, act)
  else
    # Estamos en node2, llamada remota a node1
    :rpc.call(primary, PokemonBattle.Batalla, :send_action, [room_id,
username, act])
  end
end
```

## 5.4 Monitoreo de nodos y detección de desconexión

El sistema monitorea si un nodo del cluster se desconecta. Si un jugador pierde la conexión durante una batalla, el nodo detecta el evento `:nodedown` y activa un temporizador de 15 segundos. Si el jugador no reconecta en ese tiempo, pierde la batalla automáticamente:

```
def init(opts) do
  :net_kernel.monitor_nodes(true) # Activar monitoreo de nodos
  ...
end

def handle_info({:nodedown, node}, state) do
  # Identificar qué jugador estaba en ese nodo
  disconnected_user = Enum.find_value(state.pids, fn {username, pid} ->
    if node(pid) == node, do: username, else: nil
  end)
  # Activar temporizador de 15 segundos
  timer = Process.send_after(self(), {:disconnect_timeout,
disconnected_user}, 15_000)
  {:noreply, put_in(state.disconnect_timers[disconnected_user], timer)}
end
```

## 6. Motor de Combate — MotorCombate

El módulo MotorCombate es un módulo funcional puro: no tiene proceso propio, no mantiene estado. Contiene todas las funciones matemáticas del sistema de batalla. Al ser puro, es completamente testeable de forma unitaria sin necesidad de iniciar ningún proceso.

### 6.1 Fórmula de daño

La fórmula implementada sigue la especificación del documento del proyecto:

```
daño_base = trunc((poder × (ataque_atacante / defensa_defensora)) / 5 + 2)
daño_final = trunc(daño_base × efectividad × stab × factor_aleatorio)
daño_mínimo = 1 (nunca puede ser 0 un ataque exitoso)
```

```
def calculate_damage(move, attacker, defender) do
  power      = move["base_power"]
  eff        = effectiveness(move["type"], defender["types"])
  stab_bonus = stab(move["type"], attacker["types"])
  rand_factor = 0.85 + :rand.uniform() * 0.15
  base_damage = trunc((power * (attacker["attack"] / defender["defense"]))
    / 5 + 2)
  max(trunc(base_damage * eff * stab_bonus * rand_factor), 1)
end
```

### 6.2 Efectividad de tipos

La tabla de efectividad define relaciones de fortaleza entre tipos. La debilidad es automáticamente la relación inversa. Si el defensor tiene 2 tipos, los modificadores se multiplican:

| Tipo atacante | Fuerte contra (×2.0)   | Débil contra (×0.5)  |
|---------------|------------------------|----------------------|
| Fire          | Grass, Ice, Bug        | Water, Rock, Ground  |
| Water         | Fire, Rock, Ground     | Grass, Electric      |
| Grass         | Water, Rock, Ground    | Fire, Ice, Bug       |
| Electric      | Water, Flying          | ninguno definido     |
| Rock          | Fire, Ice, Flying, Bug | Water, Grass, Ground |

```
@advantages %{
  "Fire"    => ["Grass", "Ice", "Bug"],
  "Water"   => ["Fire", "Rock", "Ground"],
  ...
}
```

```
def effectiveness(move_type, defender_types) do
  Enum.reduce(defender_types, 1.0, fn def_type, acc ->
    acc * modifier(move_type, def_type)
  end)
end
```

## 6.3 STAB — Same Type Attack Bonus

Si el tipo del movimiento coincide con alguno de los tipos del Pokémon atacante, el daño recibe una bonificación del 50% adicional. Esto incentiva usar movimientos del mismo tipo que el Pokémon:

```
def stab(move_type, attacker_types) do
  if move_type in attacker_types, do: 1.5, else: 1.0
end
```

## 6.4 Ejemplo de cálculo real

Pikachu (ataque: 63, tipo Electric) usa thunderbolt (poder: 90) contra Squirtle (defensa: 70, tipo Water):

| Variable    | Valor | Explicación                                                                     |
|-------------|-------|---------------------------------------------------------------------------------|
| efectividad | 2.0   | Electric es fuerte contra Water                                                 |
| stab        | 1.5   | thunderbolt es Electric, coincide con tipo de Pikachu                           |
| rand_factor | 0.92  | valor aleatorio sorteado en [0.85, 1.00]                                        |
| daño_base   | 18    | $\text{trunc}((90 \times (63/70)) / 5 + 2) = \text{trunc}(16.2 + 2) = 18$       |
| daño_final  | 49    | $\text{trunc}(18 \times 2.0 \times 1.5 \times 0.92) = \text{trunc}(49.68) = 49$ |

## 7. Sistema de Sobres y Economía

El sistema de economía gira alrededor de los sobres Pokémon. Los jugadores ganan monedas en batalla, las usan para comprar sobres, y al abrir los sobres obtienen Pokémon con estadísticas y movimientos aleatorios que dependen de su rareza.

### 7.1 Flujo completo de un sobre

- El jugador ejecuta `buy_pack basic` (cuesta 100 monedas) o `buy_pack advanced` (250 monedas).
- El sistema descuenta las monedas y agrega el sobre a la lista de sobres pendientes del entrenador.
- El jugador ejecuta `open_pack last` para abrir el último sobre adquirido.
- Para cada uno de los 3 Pokémon del sobre: se elige especie aleatoria del catálogo, se sortea rareza según probabilidades, se calcula `factor_rareza` aleatorio dentro del rango de esa rareza, se calculan estadísticas y se asignan 4 movimientos.
- Los 3 Pokémon se agregan al inventario del entrenador y se persisten en `trainers.json`.

### 7.2 Cálculo de estadísticas por rareza

El `factor_rareza` es un porcentaje aleatorio dentro de un rango según la rareza del Pokémon. Se aplica igual a todas sus estadísticas:

| Rareza | Rango <code>factor_rareza</code> | Prob. básico | Prob. avanzado | Fórmula                                            |
|--------|----------------------------------|--------------|----------------|----------------------------------------------------|
| common | 2% – 8%                          | 70%          | 40%            | <code>stat = round(base × (1 + factor/100))</code> |
| rare   | 10% – 20%                        | 25%          | 45%            | <code>stat = round(base × (1 + factor/100))</code> |
| epic   | 25% – 40%                        | 5%           | 15%            | <code>stat = round(base × (1 + factor/100))</code> |

### 7.3 Asignación de movimientos

Cada Pokémon recibe exactamente 4 movimientos al salir del sobre, siguiendo estas reglas en orden:

- Se eligen 2 movimientos del pool del tipo (o tipos) de la especie del Pokémon.
- Se eligen 2 movimientos adicionales de cualquier tipo del pool global.
- No puede haber movimientos repetidos.
- Los movimientos son permanentes: no se pueden cambiar después de asignados.

```
defp assign_moves(types) do
```

```
pool = Persistencia.read_moves()

# Paso 1: movimientos del tipo de la especie
type_moves = types
  |> Enum.flat_map(fn type -> Map.get(pool, type, []) end)
  |> Enum.uniq() |> Enum.shuffle() |> Enum.take(2)

# Paso 2: movimientos complementarios de cualquier tipo
extra_moves = pool |> Map.values() |> List.flatten()
  |> Enum.reject(fn m -> m in type_moves end)
  |> Enum.shuffle() |> Enum.take(2)

type_moves ++ extra_moves
end
```

## 8. Sistema de Intercambio en Tiempo Real

El intercambio de Pokémon entre entrenadores ocurre exclusivamente en tiempo real: ambos jugadores deben estar conectados simultáneamente. Cada sala de intercambio es un proceso GenServer independiente supervisado por el DynamicSupervisor, lo que garantiza aislamiento.

### 8.1 Flujo del intercambio

| Paso | Quién | Comando                 | Qué ocurre                                                           |
|------|-------|-------------------------|----------------------------------------------------------------------|
| 1    | Ana   | create_trade_room       | Crea sala, obtiene código TR-XXXX, se une automáticamente con su PID |
| 2    | Luis  | join_trade_room TR-XXXX | Se une a la sala, su PID queda registrado                            |
| 3    | Ana   | offer_pokemon <id>      | Propone un Pokémon, ambas terminales ven la oferta                   |
| 4    | Luis  | offer_pokemon <id>      | Propone su Pokémon, ambas terminales ven ambas ofertas               |
| 5    | Ana   | confirm_trade           | Confirma, queda esperando con receive hasta que Luis confirme        |
| 6    | Luis  | confirm_trade           | Confirma, el intercambio se ejecuta, ambos reciben su Pokémon        |

### 8.2 Notificaciones al jugador que espera

El jugador que confirma primero no queda bloqueado en el prompt. Su proceso entra en un receive con timeout que espera el mensaje `{:trade_completed, pokemon}`. Cuando el segundo jugador confirma, el GenServer envía ese mensaje vía PID a ambos:

```
defp wait_for_trade_completion(session) do
  receive do
    {:trade_completed, received} ->
      # Actualizar inventario y persistir
      update_inventory_after_trade(session, offered, received)

    {:battle_event, msg} ->
      IO.puts(msg)
      wait_for_trade_completion(session)
  after
    30_000 -> IO.puts("Trade timed out.")
               %{session | trade_room: nil}
  end
end
```



## 8.3 Enrutamiento entre nodos

Dado que el GenServer de Intercambio vive en node1, todos los comandos de trade desde node2 se enrutan con :rpc.call igual que las batallas:

```
result = route_to_primary(  
  fn -> Intercambio.join(code, trainer, my_pid) end,  
  fn -> :rpc.call(primary, PokemonBattle.Intercambio, :join, [code,  
    trainer, my_pid]) end  
)
```

## 9. CRUD Completo de Equipos

El sistema implementa un CRUD completo para la gestión de equipos del entrenador. Los equipos son agrupaciones de 1 a 3 Pokémon del inventario que se pueden cargar para batalla. Toda la lógica está en `GestorEntrenadores` y opera sobre datos inmutables, retornando el `trainer` actualizado sin persistir (el Servidor se encarga de persistir).

| Operación | Función                                 | Comando CLI                                             | Validaciones                                    |
|-----------|-----------------------------------------|---------------------------------------------------------|-------------------------------------------------|
| CREATE    | <code>create_team/3</code>              | <code>create_team &lt;nombre&gt; &lt;ids&gt;</code>     | Nombre único, 1-3 Pokémon, IDs en inventario    |
| READ      | <code>list_teams/1</code>               | <code>list_teams</code>                                 | Ninguna, solo muestra                           |
| READ      | <code>get_team/2</code>                 | <code>show_team &lt;nombre&gt;</code>                   | Nombre debe existir                             |
| UPDATE    | <code>rename_team/3</code>              | <code>rename_team &lt;viejo&gt; &lt;nuevo&gt;</code>    | Viejo debe existir, nuevo no debe existir       |
| UPDATE    | <code>add_pokemon_to_team/3</code>      | <code>add_to_team &lt;nombre&gt; &lt;id&gt;</code>      | Equipo < 3, Pokémon en inventario, no duplicado |
| UPDATE    | <code>remove_pokemon_from_team/3</code> | <code>remove_from_team &lt;nombre&gt; &lt;id&gt;</code> | Pokémon en equipo, equipo tiene > 1 Pokémon     |
| DELETE    | <code>delete_team/2</code>              | <code>delete_team &lt;nombre&gt;</code>                 | Nombre debe existir                             |

### 9.1 Ejemplo de agregar Pokemon a equipo existente

```
def add_pokemon_to_team(trainer, team_name, pokemon_id) do
  case Enum.find(trainer["teams"], fn t -> t["name"] == team_name end) do
    nil -> {:error, "Team not found"}
    team ->
      cond do
        length(team["ids"]) >= 3 ->
          {:error, "Team is already full (3/3)"}
        to_string(pokemon_id) in Enum.map(team["ids"], &to_string/1) ->
          {:error, "Pokemon already in this team"}
        not Enum.any?(trainer["inventory"], fn p ->
          to_string(p["id"]) == to_string(pokemon_id) end) ->
          {:error, "Pokemon not found in inventory"}
        true ->
          updated_teams = Enum.map(trainer["teams"], fn t ->
            if t["name"] == team_name,
              do: Map.put(t, "ids", t["ids"] ++ [to_string(pokemon_id)]),
              else: t
            end)
          {:ok, Map.put(trainer, "teams", updated_teams)}
      end
  end
end
```

```
end  
end
```

## 10. Persistencia de Datos

Toda la información del sistema persiste entre sesiones en archivos JSON. El módulo Persistencia centraliza todas las operaciones de lectura y escritura, facilitando el mantenimiento y el cambio futuro de backend de almacenamiento.

| Archivo            | Contenido                                                                | Cuándo se actualiza                                                   |
|--------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------|
| data/trainers.json | Lista completa de entrenadores con inventario, monedas, equipos y sobres | Al comprar/abrir sobre, ganar batalla, intercambiar, modificar equipo |
| data/pokemon.json  | Catálogo de especies base con tipos y estadísticas base                  | Solo lectura — editado manualmente para agregar especies              |
| data/moves.json    | Pool de movimientos organizados por tipo                                 | Solo lectura — editado manualmente para agregar movimientos           |
| data/shop.json     | Tipos de sobre, precios y probabilidades de rareza                       | Solo lectura — editado manualmente para cambiar economía              |
| data/battles.log   | Historial de batallas con fecha, ganador, turnos, nodo y duración        | Al terminar cada batalla (modo append)                                |

### 10.1 Patrón de lectura y escritura

```
# Lectura segura – retorna [] si el archivo no existe
def read_trainers do
  case File.read(@trainers_path) do
    {:ok, content} -> Jason.decode!(content)
    {:error, _}    -> []
  end
end
```

```
# Escritura completa – sobrescribe el archivo
def save_trainers(trainers) do
  content = Jason.encode!(trainers, pretty: true)
  File.write!(@trainers_path, content)
end
```

```
# Append al log de batallas
def log_battle(info) do
  line = "#{info.date} | #{info.player1} vs #{info.player2} | " <>
        "Winner: #{info.winner} | Turns: #{info.turns} | " <>
        "Node: #{info.node} | Duration: #{info.duration}\n"
```

```
File.write!(@battles_log, line, [:append])  
end
```

## 10.2 Jason — serialización JSON

La librería Jason (declarada en mix.exs como dependencia) maneja la serialización y deserialización de datos entre Elixir y JSON. Es significativamente más rápida que otras alternativas como Poison y es el estándar de facto en el ecosistema Elixir.

## 11. Sistema de Evolución (Extra)

Como funcionalidad adicional al alcance mínimo del proyecto, se implementó un sistema de evolución automática de Pokémon. Después de cada victoria en batalla, el sistema revisa si el Pokémon activo del ganador acumuló suficientes victorias para evolucionar.

### 11.1 Cadena de evolución recursiva

La función `build_chain/3` recorre la cadena de evolución de una especie de forma recursiva, acumulando los estadíos hasta llegar a la forma final (que no tiene evolución definida):

```
defp build_chain(catalog, species, acc) do
  base = catalog[species]
  new_acc = acc ++ [species]

  case base["evolution"] do
    nil -> new_acc # Caso base: forma final
    next -> build_chain(catalog, next, new_acc) # Recursión
  end
end
```

### 11.2 Escalado de estadísticas al evolucionar

Al evolucionar, las estadísticas del Pokémon se escalan proporcionalmente usando la razón entre las bases de la nueva forma y la anterior. Esto preserva la ventaja de rareza que tenía el Pokémon original:

```
new_attack = round(old_attack * (new_base["base_attack"] /
base["base_attack"]))
new_defense = round(old_defense * (new_base["base_defense"] /
base["base_defense"]))
new_speed = round(old_speed * (new_base["base_speed"] /
base["base_speed"]))
```

### 11.3 Cadenas de evolución implementadas

| Etapas 1   | Victorias | Etapas 2   | Victorias | Etapas 3  |
|------------|-----------|------------|-----------|-----------|
| Charmander | 3         | Charmeleon | 6         | Charizard |
| Squirtle   | 3         | Wartortle  | 6         | Blastoise |
| Bulbasaur  | 3         | Ivysaur    | 6         | Venusaur  |
| Pikachu    | 3         | Raichu     | —         | —         |
| Geodude    | 3         | Graveler   | 6         | Golem     |
| Pidgey     | 3         | Pidgeotto  | 6         | Pidgeot   |

| Etapa 1 | Victorias | Etapa 2 | Victorias | Etapa 3 |
|---------|-----------|---------|-----------|---------|
| Machop  | 3         | Machoke | 6         | Machamp |
| Gastly  | 3         | Haunter | 6         | Gengar  |

## 12. Pruebas con ExUnit

El proyecto incluye una suite de pruebas unitarias usando ExUnit, el framework de testing integrado en Elixir. Las pruebas son completamente independientes del sistema en ejecución: no requieren que los GenServers estén corriendo porque prueban funciones puras.

| # | Requisito del documento  | Qué se prueba                                                                                                   | Resultado |
|---|--------------------------|-----------------------------------------------------------------------------------------------------------------|-----------|
| 1 | Cálculo de daño por tipo | Electric vs Water > daño neutro; Water vs Electric <= neutro; daño mínimo = 1                                   | Pass      |
| 2 | Orden por velocidad      | Pikachu (104) más rápido que Squirtle (46); initialize_team pone HP=100; apply_damage no baja de 0              | Pass      |
| 3 | Monedas al terminar      | add_coins suma a saldo actual y acumulado; ganador +100; perdedor +30                                           | Pass      |
| 4 | Apertura de sobres       | create_instance da 4 movimientos; ≥2 del tipo de la especie; dueño_original correcto; buy_pack descuenta precio | Pass      |
| 5 | Intercambio de Pokemon   | Pokemon conserva id, rareza y original_owner; inventarios se actualizan correctamente                           | Pass      |

Para ejecutar las pruebas:

```
mix test
mix test --trace # Con detalle de cada test
```

### 12.1 Por qué las pruebas no necesitan GenServers

Las funciones más importantes del sistema (cálculo de daño, efectividad, estadísticas de sobres, gestión de equipos) son funciones puras en módulos sin estado. Esto es una ventaja directa del paradigma funcional: se puede probar la lógica de negocio completamente aislada de la infraestructura de procesos y distribución.



## 13. Referencia Completa de Comandos

### 13.1 Sesión y perfil

| Comando             | Descripción                                                                    |
|---------------------|--------------------------------------------------------------------------------|
| login <user> <pass> | Login o registro automático si no existe la cuenta                             |
| logout              | Cierra la sesión actual                                                        |
| profile             | Muestra victorias, monedas, monedas acumuladas y cantidad de Pokemon           |
| leaderboard         | Clasificación global ordenada por victorias (desempate por monedas acumuladas) |
| inventory           | Inventario completo con ids, estadísticas y movimientos de cada Pokemon        |

### 13.2 Economía

| Comando              | Descripción                                                         |
|----------------------|---------------------------------------------------------------------|
| shop                 | Lista tipos de sobre con precio y probabilidades de rareza          |
| buy_pack <tipo>      | Compra un sobre: basic (100 coins) o advanced (250 coins)           |
| open_pack <id  last> | Abre un sobre y muestra los 3 Pokemon obtenidos con sus movimientos |

### 13.3 Equipos — CRUD

| Comando                        | Descripción                                           |
|--------------------------------|-------------------------------------------------------|
| create_team <nombre> <ids>     | Crea equipo con 1-3 Pokemon (ids separados por coma)  |
| show_team <nombre>             | Detalle de un equipo con estadísticas de cada Pokemon |
| list_teams                     | Lista todos los equipos guardados con sus IDs         |
| use_team <nombre>              | Carga equipo para la siguiente batalla                |
| add_to_team <nombre> <id>      | Agrega un Pokemon al equipo (máximo 3)                |
| remove_from_team <nombre> <id> | Quita un Pokemon del equipo (mínimo 1 debe quedar)    |
| rename_team <viejo> <nuevo>    | Renombra un equipo                                    |
| delete_team <nombre>           | Elimina el equipo completo                            |

### 13.4 Batalla

| Comando                | Descripción                                                           |
|------------------------|-----------------------------------------------------------------------|
| create_room <segundos> | Crea sala de espera con tiempo de turno configurable                  |
| list_rooms             | Lista salas disponibles en estado waiting                             |
| join_room <id>         | Se une a una sala (requiere use_team previo)                          |
| start_battle <id>      | Inicia la batalla (requiere 2 jugadores en la sala)                   |
| attack <movimiento>    | Usa un movimiento del Pokemon activo en el turno actual               |
| switch <pokemon_id>    | Cambia al Pokemon indicado del equipo (o forzado si el activo faintó) |
| surrender              | Rendición inmediata, el rival gana                                    |

## 13.5 Intercambio

| Comando                  | Descripción                                                           |
|--------------------------|-----------------------------------------------------------------------|
| create_trade_room        | Crea sala de intercambio y muestra el código para compartir           |
| join_trade_room <codigo> | Se une a la sala con ese código                                       |
| offer_pokemon <id>       | Ofrece un Pokemon del inventario para intercambiar                    |
| confirm_trade            | Confirma la oferta (cuando ambos confirman se ejecuta el intercambio) |
| cancel_trade             | Cancela y cierra la sala para ambos participantes                     |

## 13.6 Cluster

| Comando                  | Descripción                                                |
|--------------------------|------------------------------------------------------------|
| connect_node <node@host> | Conecta este nodo a otro nodo Erlang del cluster           |
| list_nodes               | Muestra todos los nodos conectados al cluster              |
| cluster_info             | Resumen: nodo actual, cantidad de nodos y batallas activas |
| evolution <especie>      | Muestra la cadena de evolución completa de una especie     |

## 14. Guía de Inicio Rápido

### 14.1 Arrancar el sistema en 2 terminales

Terminal 1 — Nodo primario (ana):

```
iex --sname node1 --cookie pokemonbattle -S mix
PokemonBattle.Servidor.start()
login ana 1234
```

Terminal 2 — Nodo secundario (luis):

```
iex --sname node2 --cookie pokemonbattle -S mix
PokemonBattle.Cluster.connect(:"node1@TUNOMBRE")
PokemonBattle.Servidor.start()
login luis 1234
```

### 14.2 Flujo completo de batalla

|                                                        |                            |
|--------------------------------------------------------|----------------------------|
| # Ana (Terminal 1)                                     | # Luis (Terminal 2)        |
| open_pack last                                         | open_pack last             |
| create_team eq 111,222,333                             | create_team eq 444,555,666 |
| use_team eq                                            | use_team eq                |
| create_room 999                                        |                            |
| join_room S-0001                                       | join_room S-0001           |
|                                                        | start_battle S-0001        |
| attack surf                                            | attack megahorn            |
| # (espera a luis)                                      | # (espera a ana)           |
| # → turno resuelto en ambas terminales simultáneamente |                            |

### 14.3 Flujo completo de intercambio

|                                              |                         |
|----------------------------------------------|-------------------------|
| # Ana (Terminal 1)                           | # Luis (Terminal 2)     |
| create_trade_room                            |                         |
| # → código TR-XXXX                           | join_trade_room TR-XXXX |
| offer_pokemon 111                            | offer_pokemon 444       |
| confirm_trade                                | confirm_trade           |
| # → ambos reciben su Pokemon automáticamente |                         |

### 14.4 Verificar distribución

```
# Desde cualquier terminal:
PokemonBattle.Cluster.info()
PokemonBattle.SupervisorBatallas.list_battles()
PokemonBattle.SupervisorBatallas.battle_count()
```